

# Full-Stack Developer Journey — Source of Truth v3

Taha Jamaly · v3 · 18 August 2026 · Journey started April 2026 · Replaces v2 (July 2026) and all earlier plan documents

**What this document is.** The *what, why and where* of the journey: curriculum, links, projects, standards, playbooks, positioning. It works hand-in-hand with the tracker ( `journey-t-tracker` , deployed from your GitHub), which owns the *when and how much*: sessions, weeks, and the live projected dates. This document changes rarely; the tracker changes daily.

**Companion repo:** [github.com/DevJamaly/guides-and-cheatsheets](https://github.com/DevJamaly/guides-and-cheatsheets) — your own guides and cheat sheets. Every time you finish a phase, drop a one-page cheat sheet there. That repo becomes interview revision material *and* public footprint.

**Superseded:** the v2 timeline (dates), Projects 3–5 of v2 (Quiz/Whiteboard, Content SaaS), all old architecture docs ( `*_ARCHITECTURE.md` — to be rewritten fresh when each project starts). `Projects_v3.md` remains the authoritative project definition; its content is folded into §4 here.

## 0. The operating system (read this page when tempted to deviate)

**Target.** Hired as a full-stack developer with backend depth — UAE product company, or remote US/EU product company.

**Positioning: a mid-level engineer changing stacks — never junior.** Eight years shipping real-time systems is the asset; the web stack is the delta.

**The one rule.** *Understand → Build → Move on.* You feel ready by building, not by watching more videos.

**The method.** Sessions, not hours. One session = one protected sitting of ~2 hours. A normal week is four sessions (two weeknights + one weekend block). Videos at 2×. Type all code manually. Complete all exercises. Phone in another room. Commit at the end of every session.

**The three modes (declared every Sunday in the tracker, from your real calendar):**

| Mode     | Sessions | When                                                                                            |
|----------|----------|-------------------------------------------------------------------------------------------------|
| DEFAULT  | 4 (~8 h) | A normal week. This should be most weeks. It's a floor: finish early → <i>Pull next</i> .       |
| REDUCED  | 2 (~4 h) | Busy week (events, travel, exhaustion). Keep the active track warm; fill fragments with Tier-1. |
| FALLBACK | 0        | No capacity. One ≤30-min keep-warm touch so the thread doesn't snap.                            |

**The three tiers (what to do with the time you actually have):**

| Tier              | Situation                       | Do                                                                                                                               |
|-------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 1 • Passive       | On-site, transit, dead evenings | Nasser system-design videos · Karpathy / 3Blue1Brown · cert lectures · TS Handbook · Testing Trophy · docs reading               |
| 2 • Interruptible | 30–60 min pockets               | Cert practice exams · small course exercises · react.dev reading · a blog paragraph · LeetCode Easy (from interview-prep window) |
| 3 • Protected     | Blocked 2 h+                    | The active course section · project code · portfolio build. This is the only tier that moves the projection meaningfully.        |

**The flags.** Active track untouched for 10+ days = productive procrastination — stop and do the hard thing. No daily drill for 3 days = the habit that compounds is slipping. The tracker raises both.

**Cut order if compression is ever forced:** StockSense's AI search → Avatar's optional pieces → *never* NFC depth, Commerce's ledger/outbox, or the Avatar's core.

## 1. Where you are — August 2026

| Done                                                                                     | In progress                                                                                                | Next                                                                              |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Git & GitHub · Web foundations · JavaScript (Jonas, Forkify shipped) · HTML/CSS/Tailwind | <b>React + TypeScript</b> — Jonas Ultimate React, part 8 complete; part 9 (usePopcorn) is the next session | Project 1 (Portfolio) → Backend reading → Node/Express → Databases → NFC Platform |

Public footprint so far: dev.to account created, zero posts. First post is a one-session task in the tracker ( `p1-6` ) — write it *before* the portfolio ships; dev.to is home until tjamaly.com goes live, then it becomes the cross-post.

## 2. The learning spine — courses, focus, done-when

One line per phase; the tracker holds the session-sized tasks (IDs in brackets so you can find them). Completed phases stay here as revision references — return to them before interviews.

### 2.1 ✓ Git & GitHub — done

- **Course:** [The Git & GitHub Bootcamp](#) — Colt Steele (Udemy)
- **Revisit before interviews:** interactive rebase, GitFlow vs trunk-based, [Conventional Commits](#) (in use on every repo).

### 2.2 ✓ Web foundations — done

- **Reading:** [How the web works](#) (MDN) · [HTTP overview](#) (MDN) · [How browsers work](#) (web.dev) · [What is DNS](#) (Cloudflare) · [HTTP vs HTTPS](#) (Cloudflare)
- **Revisit:** "what happens when you type a URL" and "how does the browser render a page" — guaranteed interview questions.

### 2.3 ✓ JavaScript — done

- **Course:** [The Complete JavaScript Course](#) — Jonas Schmedtmann (Udemy) · Reference: [javascript.info](#)
- **Revisit:** closures, `this`, prototypes, event loop, promises/async — the interview core. Forkify is your "explain every async call" drill.

### 2.4 ✓ HTML · CSS · Tailwind — done

- [Kevin Powell](#) (YouTube) · [CSS-Tricks Flexbox guide](#) · [CSS-Tricks Grid guide](#) · [Tailwind docs](#) · [MDN Accessibility](#)
- **Revisit:** a11y basics appear in take-home rubrics.

### 2.5 ► React + TypeScript — active · make-or-break · event season

- **Course:** [The Ultimate React Course](#) — Jonas Schmedtmann (Udemy) — one tracker task per section ( `react-5` ... `react-24` ), including the Next.js part (RSC, App Router, server actions) because Project 1 and Project 2 are Next.js.
- **Parallel Tier-1:** [TypeScript Handbook](#) ( `react-25/26` ) · [react.dev](#) alongside the course · [Kent C. Dodds](#) — [The Testing Trophy](#)
- **Testing intro:** 3–5 component tests with [React Testing Library](#) + [Vitest](#) on one course app ( `react-27` ).
- **Backup if Jonas isn't clicking:** [Bob Ziroll](#) — [Learn React](#) (Scrimba, free).
- **Unity map:** Component = MonoBehaviour · Props = serialized fields · `useState` = member vars · `useEffect` = `Start()+OnDestroy()` · custom hooks = utility scripts · Context = game managers.
- **Hard rule:** minimum two protected sessions per week even in peak event weeks. If this phase drifts, everything shifts.
- **Done when:** an interactive app with fetched data, loading/error states and routing — from scratch, in TypeScript, no tutorial open.

## 2.6 Project 1 — Portfolio site ( p1-\*, ~11 sessions)

- Next.js static → Vercel/GitHub Pages → [tjamaly.com](https://tjamaly.com) · MDX blog + case-study template · Lighthouse ≥95 enforced in CI · your first repo with the full CI template (lint → typecheck → test → deploy).
- **Docs:** [Next.js](#) · [MDX](#) · [Vercel](#) · [GitHub Actions](#) · [Lighthouse CI](#)
- **Hard cap:** 2–3 weeks. Don't over-engineer; it's the writing home and the CI origin, not the centrepiece.

## 2.7 Backend reading ( be-\*, 3 sessions, all Tier 1)

- [roadmap.sh/backend](https://roadmap.sh/backend) — Internet & HTTP sections only · Hussein Nasser: [reverse proxy vs load balancer](#) · HTTP/1.1 vs 2 vs 3 · TCP vs UDP · TLS handshake.

## 2.8 Node.js + Express + Testing ( nd-\*, ~18 sessions)

- **Course:** [Node.js Bootcamp](#) — [Jonas Schmedtmann](#) (Udemy) — Express, auth and API design are stack-agnostic; follow the Mongo sections for the teaching only, and **build the real data layer in Postgres + Prisma in parallel** ( nd-7/8 ).
- **Focus:** event loop & non-blocking I/O · routing/middleware · REST design + error handling · JWT/sessions/roles · rate limiting, helmet, OWASP Top 10 · deploy.
- **Testing:** [Jest](#) + [Supertest](#) integration tests · [Zod](#) on every input — validation is the habit that screams "production engineer".
- **Supplement:** [The Odin Project](#) — [Databases \(PostgreSQL + Prisma\)](#)
- **Done when:** a secure, rate-limited, Zod-validated, Jest-tested REST API from a blank folder.

## 2.9 Databases ( db-\*, ~9 sessions, parallel with Node — not before)

- **Course:** [Fundamentals of Database Engineering](#) — [Hussein Nasser](#) (Udemy) — indexes, transactions, ACID, isolation, partitioning/sharding overview.
- **Docs & reading:** [Prisma](#) · [PostgreSQL Tutorial](#) · [Use The Index, Luke](#) · [Supabase docs](#) (RLS, auth) for the React course project.
- **Done when:** you can design the NFC platform's normalised schema and explain where its indexes go and why ( db-7 ).

## 2.10 Projects 2–5 — see §4. Cloud certs, system design, AI literacy — see §3.

---

# 3. Always-on threads (passive pools in the tracker — log units in *Threads*)

---

## 3.1 System design — your #1 interview edge

- **Now → exhaust the free content:** [Hussein Nasser](#) (YouTube) — perfect Tier-1 for events and transit.
- **Interview-prep window:** [systemdesignschool.io](https://systemdesignschool.io) → [Alex Xu](#) — [System Design Interview Vol. 1](#) (ByteByteGo).
- **The answer template you're building toward:** *"I've built the monolith-scale version — here's the pattern, here's when I'd reach for the distributed version, here's why I didn't."* Backed by running, tested code (§4).

## 3.2 AI & LLM literacy — foundations → applied → workflow (almost all Tier 1)

| Topic                                   | Resource                                                                                                                  |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| How LLMs work                           | <a href="#">Karpathy</a> — <a href="#">Intro to LLMs</a> · <a href="#">Karpathy</a> — <a href="#">Deep Dive into LLMs</a> |
| Transformers intuition                  | <a href="#">3Blue1Brown</a> — <a href="#">Neural networks / Transformers series</a>                                       |
| Prompt engineering                      | <a href="#">Anthropic prompting docs</a> · <a href="https://promptingguide.ai">promptingguide.ai</a>                      |
| Embeddings & RAG                        | <a href="#">pgvector</a> readme + one solid RAG explainer — before the Avatar                                             |
| Structured outputs, tool use, streaming | <a href="#">Anthropic API docs</a> · <a href="#">OpenAI API docs</a> — before Projects 3–5                                |

| Topic                   | Resource                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------|
| MCP awareness           | <a href="https://modelcontextprotocol.io">modelcontextprotocol.io</a> — hot interview topic               |
| Staying current (light) | <a href="https://roadmap.sh/ai-engineer">Simon Willison · roadmap.sh/ai-engineer</a> for orientation only |

**Scope guard:** you integrate AI; you are not an ML engineer. No TensorFlow, no model training, no maths detours. The bar: explain how an LLM works at Karpathy-video depth, and ship streaming + RAG + validated LLM features in production.

### 3.3 Cloud certifications (1 hr/day, interview-prep window; both are Tier-1/2 filler until then)

| Cert                           | Course                                  | Practice                                           | Free official                     |
|--------------------------------|-----------------------------------------|----------------------------------------------------|-----------------------------------|
| AWS Cloud Practitioner CLF-C02 | <a href="#">Stephane Maarek (Udemy)</a> | <a href="#">Jon Bonso / Tutorials Dojo (Udemy)</a> | <a href="#">AWS Skill Builder</a> |
| Azure Fundamentals AZ-900      | <a href="#">Scott Duffy (Udemy)</a>     | <a href="#">MeasureUp</a>                          | <a href="#">Microsoft Learn</a>   |

Deeper certs (SAA, CKA, Terraform) come after hire — employers often pay.

### 3.4 Public footprint — GitHub + writing (start now, compounds)

- **GitHub:** commit every session ( `counters` → `commits` ) · Conventional Commits from day one · feature branches + PRs on your own projects from Project 2 · profile README (pitch + live links), pin best 4–6 repos, archive junk — groom fully before soft applications.
- **Guides repo:** [DevJamaly/guides-and-cheatsheets](#) — one cheat sheet per finished phase (JS event loop, React hooks, TS generics, SQL indexing, HTTP/auth, Stripe webhooks...). Written in your words, it's revision material *and* proof of understanding.
- **Blog:** [dev.to](#) now → portfolio blog is home once P1 ships, dev.to cross-posts, LinkedIn teaser always. Cadence: 1 post/month + 1 deep writeup per project. Short is fine (600–1000 words); consistency beats polish.
- **Pillars:** (1) Unity → Web translations — zero competition · (2) build logs & architecture decisions with trade-offs · (3) AI validation stories — "the AI wrote this, here's the bug it hid, here's the test that caught it" · (4) debugging war stories.
- **First post (do it now, p1-6):** *"From Unity coroutines to JavaScript async/await — a game developer learns the event loop."* You know this cold.

## 4. Projects — build order, depth, and what each proves

**Thesis (say it in interviews): fundamentals first, AI forward.** Fewer services, deeper systems, more things built by hand *after* first using the library — every abstraction understood from underneath. Hosting ladder: static → serverless → single container → VPS monolith + workers → multi-service.

### The learn-then-build-beneath ladder

| Concern  | First: use the tool (NFC)        | Then: build what it did (Commerce / StockSense)                                                                |
|----------|----------------------------------|----------------------------------------------------------------------------------------------------------------|
| Auth     | better-auth, sessions            | argon2id, own session tokens, cookie flags, CSRF, timing-safe compare, reset-token lifecycle, login throttling |
| Payments | Stripe Checkout + Billing Portal | Raw Payment Intents (own UI, 3DS/SCA, every state) + <b>own double-entry ledger</b> + reconciliation vs Stripe |
| Database | Prisma on managed Neon           | Self-managed Postgres: raw SQL paths, EXPLAIN ANALYZE, isolation levels, bulk ingestion, big-table migrations  |
| Infra    | Vercel PaaS                      | DigitalOcean VPS: nginx, systemd, own deploy scripts, backups, monitoring                                      |
| Jobs     | Vercel cron                      | BullMQ + DLQ + transactional outbox + at-least-once semantics, named and tested                                |

### 4.1 Project 1 • Portfolio site — frontend • 2–3 wk cap

Next.js static → tjamaly.com. Case studies (incl. professional interactive work), MDX blog, Lighthouse ≥95 in CI. **Proves:** polished frontend, deployment, SEO, the CI/CD template origin.

### 4.2 Project 2 • NFC / QR Card Platform ★ centrepiece — soft applications begin on ship ( p2-\*, ~27 sessions)

Multi-tenant SaaS: cards → public profiles → scan analytics → Stripe tiers. Library auth ([better-auth](#)) and [Stripe Checkout](#) are *deliberate* — first full-stack build in a hard window carries library risk, not hand-rolled risk. Stack: Next.js · TS · Postgres/[Prisma](#) on [Neon](#) · Redis · Stripe · Zod. - Reliability adds: `/healthz` + external uptime monitor · nightly reconciliation cron (Stripe subscription state vs DB) · ADR naming Stripe webhooks **at-least-once** → idempotency table. - Full toolkit applied: Jest + RTL + [Playwright](#) in CI, Zod everywhere, AI code-review Action, security rigour stated in README. - **Never-cut test:** webhook idempotency. **Proves:** auth flows, schema design, payments integration, webhook idempotency, serverless architecture, product thinking.

### 4.3 Project 3 • StockSense Deep — databases • concurrency • scale ( p3-\*, ~17 sessions, || 0.6, overlaps NFC tail)

Internal inventory platform for a fictional UAE distributor ("Gulf Tyre & Parts", 3 warehouses). Real-time stock sync (Socket.io + Redis adapter), role-based mutations, append-only audit log. - **Scale is the headline:** ~500 SKUs → millions of rows via a hand-built ingestion pipeline over an open dataset (batched `COPY`, resumable checkpointed jobs, retry/backoff, checksum validation, dedup). - **DB internals as deliverables:** EXPLAIN ANALYZE index decisions with before/after timings · keyset vs offset pagination benchmark · isolation-level anomalies (lost update, phantom) reproduced in integration tests · a big-table migration without downtime, written up. - AI NL-search (tools + Zod where-builder + evals) is one *feature*, first on the cut list. - **Never-cut tests:** atomic decrement race · WS convergence · anomaly repros. **Proves:** transactions, ACID, locking, isolation, indexing at real scale, ETL, real-time sync, concurrency correctness — the Unity-multiplayer bridge.

### 4.4 Project 4 • Commerce Engine ★ second centrepiece ( p4-\*, ~22 sessions)

Customer-facing storefront for the same fictional distributor: catalog, cart, checkout, orders. Four depth pillars, each a "built it myself, underneath the library" story: 1. **Hand-rolled auth** — argon2id, own session design (generation, storage, rotation, revocation), cookie flags reasoned in an ADR, CSRF, timing-safe compares, email verification + reset-token lifecycle, login throttling. Blog: "[I used an auth library, then built one — here's what it was hiding.](#)" Refs: [OWASP Cheat](#)

Sheets · OWASP Top 10 2. **Payments engineering** — Stripe stays as card rails (know why hand-rolling card processing is not a thing), but **raw Payment Intents**: own checkout UI, 3DS/SCA, every state, refunds, disputes. Beneath it a **double-entry ledger from scratch**: debits = credits enforced in serializable transactions, idempotency keys on money mutations, auth-then-capture, refund reversals. **The money race test (parallel spends against one balance) is non-negotiable**. 3. **Async order pipeline** — transactional outbox → BullMQ workers → dead-letter queue with admin requeue → retries + backoff → compensation paths → nightly reconciliation vs Stripe. ADRs name at-least-once delivery and idempotent consumers. 4. **Self-hosted ops** — DigitalOcean droplet: self-managed Postgres + Redis, nginx + TLS, systemd, own deploy script, pg\_dump backup + tested restore, health checks, graceful degradation per dependency. - AI presence minimal by design (at most one cuttable feature). **Proves**: auth from first principles, payments + bookkeeping, distributed-reliability patterns at monolith scale, queue architecture, Linux ops.

4.5 Project 5 · AI Avatar Assistant ★ the differentiator ( p5-\*, ~19 sessions, || 0.6, parallel with Commerce tail)

Push-to-talk → streaming STT → streaming LLM → chunked TTS || viseme inference (own NeuroSync fork on Modal GPU) → lip-synced R3F avatar. <2 s end-to-end latency, benchmarked and published. pgvector memory = your RAG credential. Multi-service Docker. - **Why it survives the fundamentals purge**: streaming orchestration, backpressure, state machines, abort chains, binary protocols, latency engineering *is* core engineering. It's the direct bridge to eight years of professional avatar work and the one project no career-changer can copy. NeuroSync licence verification gates the repo going public. - **Never-cut tests**: contract tests · latency bench. **Proves**: streaming architectures, multi-service, real-time audio/graphics sync, applied AI with rigour.

Parked, post-hire: AI Presenter web version. Do not unpark.

4.6 What each project proves — no overlap

| Project      | Frontend | Auth            | DB depth       | Real-time     | Async/queues   | Payments           | Ops           | AI       |
|--------------|----------|-----------------|----------------|---------------|----------------|--------------------|---------------|----------|
| 1 Portfolio  | ✓ ✓      |                 |                |               |                |                    | CI/CD origin  | workflow |
| 2 NFC        | ✓        | library ✓       | schema ✓       |               |                | Checkout ✓         | serverless    | feature  |
| 3 StockSense | ✓        | reuse           | ✓ ✓ scale/ACID | ✓ ✓           |                |                    | container     | feature  |
| 4 Commerce   | ✓        | ✓ ✓ hand-rolled | ✓ ✓ ledger     |               | ✓ ✓ outbox/DLQ | ✓ ✓ Intents+ledger | ✓ ✓ VPS       | minimal  |
| 5 Avatar     | ✓ 3D     |                 | pgvector       | ✓ ✓ streaming | backpressure   |                    | multi-service | ✓ ✓      |

4.7 Rules for every project

- 1. Live URL, always; seeded demo data + demo credentials in the README.
- 2. **Mpty rule**: cut order + anti-goals in every arch doc; tag v1.0.0 on schedule and move on. Iteration ≠ new scope.
- 3. Final week of every project: polish and proof only — case study on tjamaly.com + blog post before moving on.
- 4. README: architecture diagram, **trade-offs section** (what you rejected and why), CI badge, honest limitations, "How I use AI" section.
- 5. No tutorial clones. No skill bars. Learning-in-progress repos stay private until ship.
- 6. Every week of every build ends deployed. Migrations are commits, reviewed like code.
- 7. Write the fresh ARCHITECTURE.md + first ADRs in week 1 of each project (the old ones are retired).

## 5. Engineering standards — the reusable template (lineage 1 → 2 → 3 → 4 → 5)

---

**Git:** trunk-based · protected `main` · short-lived `feat/*` `fix/*` `chore/*` `docs/*` · Conventional Commits from commit #1 · every change via PR with a real description · squash merge · tags + CHANGELOG at each ship. No GitFlow, no `develop`.

**CI (each repo adapts, never abandons):** `quality` (lint → typecheck → unit) · `integration` (real Postgres/Redis as ephemeral CI services — never mock the DB in integration tests) · `e2e` (Playwright golden path) · `ai-review` (Claude on every PR diff) · project-specific jobs (evals, docker build, bench). Required checks on `main`. `Husky` + `lint-staged` pre-commit; `Dependabot` on every repo.

**Testing pyramid (never cut — this is the differentiation):** many unit · some integration · few E2E. Rule of proportion: ~10–15 % of project time. Goal: credible coverage of critical paths + a green badge, not coverage vanity. Runtime validation with `Zod` (+ `React Hook Form` on the frontend); shared schemas between client and server is a senior-signal pattern.

**Docs:** ADRs for every real decision incl. rejected alternatives · `ARCHITECTURE.md` with diagrams · `CLAUDE.md` with repo conventions for AI agents.

### The Human/AI split — learning-first, non-negotiable

Rule zero: **if you can't defend a merged line in an interview, it doesn't merge.**

- **You hand-write, always:** all business logic (ledger maths, atomic decrements, auth crypto flows, where-builders, state machines, outbox/queue consumers) · **the first test of every kind** in every suite · all critical-path tests (money race, decrement race, idempotency, auth security) · every schema, migration, index decision and `EXPLAIN ANALYZE` investigation · every ADR and diagram · debugging hypotheses.
- **AI is leveraged for:** scaffolding *after* the exemplar exists · CRUD boilerplate, fixtures, seeds · PR review via `ai-review.yml` (reviewer, never author of record; findings triaged by you) · rubber-duck design sessions, docs polish, config wrangling · explaining unfamiliar territory before you implement it yourself.
- **During course phases:** AI explains but never writes your exercise code. **During projects:** AI at full speed, validated hard. Calibration: Commerce auth + ledger ≈ 90 % hand-written; portfolio UI polish AI-heavy is fine.
- **Daily driver:** `Claude Code` (or `Cursor` / `Copilot` — pick one and get fluent from the React phase on). Fluency with these tools is now a hiring requirement, not a bonus.
- **The public story ("How I use AI" in every README):** fluency + judgment + controls — evals gate prompt changes, review gates code, a human owns every decision.

---

## 6. Event season — tiers, not plans

---

Peak runs roughly **late August → mid December**. There is no plan for it; the tracker absorbs it. What you do:

- Each Sunday, count the evenings you can genuinely protect. Three or more → **DEFAULT**. One or two → **REDUCED**. Zero → **FALLBACK**. Change mid-week if reality changes; nothing you logged is lost.
- Tier-1 material lives on your phone for event floors and transit (Nasser, Karpathy, cert lectures, TS Handbook). Tier-2 for quiet hours. Tier-3 is calendared and defended.
- Minimum viable peak week: two protected sessions on the active track + whatever Tier-1/2 fits. If even that fails two weeks running, don't compress — the projection moves, and you say so at the next check-in.
- Never let a **REDUCED** month silently become a "restart from zero" month: the keep-warm touch exists precisely so the thread never snaps.

## 7. The job market you're walking into — and where you're safe

**The honest picture (mid-2026).** The market has split. Entry-level, generalist "ship CRUD from a ticket" roles are shrinking because AI does that work; senior and specialised roles — system design, reliability, security, platform, and applied AI/LLM integration — are growing, and companies say AI-integration skills are the hardest to hire for. Companies hire for three things: cut cost, ship revenue, keep systems up. Everything in this plan maps to at least one; the centrepieces map to all three.

**Where you are safe (target these titles):**

| Role family                                                                                             | Why it holds against AI                                                                                                                                | Your evidence                                                                                              |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Product / full-stack engineer with backend depth</b> at a product company                            | Owens outcomes across the stack; judgment about trade-offs, data, money and failure modes is exactly what tools don't supply                           | The ladder (\$4): NFC → Commerce. Payments, auth, schema, tests, CI                                        |
| <b>Applied AI / AI-integration engineer</b> (LLM features, RAG, evals, streaming) — <i>not</i> ML       | Hardest-to-hire skill globally; every product wants validated, cost-controlled LLM features shipped by someone who understands the systems around them | Avatar (streaming, RAG, latency), StockSense NL-search + evals, "How I use AI" READMEs, ai-review pipeline |
| <b>Real-time / interactive systems</b> (collab tools, live events tech, streaming, gaming-adjacent web) | Deep, non-commodity domain; state sync, backpressure, latency are hard-won                                                                             | Eight years of professional real-time avatar work + StockSense WS sync + Avatar                            |
| <b>Reliability / payments-leaning backend</b> in fintech, proptech, logistics, govtech                  | Correctness under money and concurrency is where AI-generated code is least trusted; regulated domains want humans who can prove it                    | Ledger, outbox, DLQ, race tests, reconciliation — running, tested code                                     |
| <b>Forward-deployed / solutions engineer</b> at an AI or SaaS company (lateral door)                    | Customer-facing + ships integrations fast + explains trade-offs; a growing role family that values domain experience                                   | Events-industry background, client delivery history, communication via blog                                |

**What to avoid:** anything titled junior; agency/CRUD churn; roles where the whole job is what a coding assistant does; pure template frontend. Also avoid over-rotating into "AI engineer" hype without the fundamentals — the thesis is *AI forward*, on top of basics, and that combination is rarer than either alone.

**Positioning (unchanged and sharpened).** *"Real-time systems engineer, eight years, now shipping on the web stack — with tested payments, auth and streaming systems in production."* Never "career switcher learning web dev." Lead with the ladder and the never-cut tests; they say mid-level without you having to.

**Markets.** UAE: fintech · proptech · logistics · healthtech · government digital transformation · events-tech (ticketing, event SaaS — your unfair domain advantage). Prefer enterprise/financial/infrastructure product companies over consumer startups. Remote US/EU product companies in parallel.

**Boards & networking.** Remote: [Arc.dev](#) · [Turing](#) · [Wellfound](#) · [We Work Remotely](#) · [Himalayas](#) · [HN Who's Hiring](#) (first Monday monthly). UAE: [Bayt](#) · [GulfTalent](#) · [LinkedIn Jobs](#) (React/TypeScript filters). 1–2 Dubai tech meetups per quarter ([meetup.com](#)) · [GITEX](#) in October — free networking during event season anyway · referrals beat cold applications in the UAE.

**When.** Soft applications open when NFC ships. Full push after the Avatar. Hiring can — and should — land in between; don't wait for all five projects.

## 8. Interview preparation — start six weeks before the soft-application date

### 8.1 The four rounds and what each wants

| Round                        | What they're testing                                   | Your prep                                                                                                               |
|------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Recruiter screen             | Story coherence, seniority signal                      | The 60-second pitch (below), salary range, notice period, why web                                                       |
| Live coding / take-home      | Fundamentals under pressure, code quality              | LeetCode Easy → Medium in JS/TS, 30 min/day · "build this component live" reps · take-home rules below                  |
| Backend / system design      | Judgment, trade-offs, failure modes                    | Walk your own systems: idempotency, outbox, ledger, race tests, WS convergence. Nasser → Alex Xu template answer (§3.1) |
| Behavioural / hiring manager | Ownership, communication, collaboration under pressure | 6–8 STAR stories from eight years: a launch, a production fire, a disagreement, a cut scope, a mentee, a mistake        |

**8.2 The 60-second pitch (memorise, then make it yours).** Eight years building real-time avatar and interactive systems for live events — latency, sync, and shipping under hard deadlines. In 2026 I moved that engineering to the web stack: TypeScript, React/Next.js, Node, Postgres. The portfolio is deliberately layered — I used auth and payments libraries in the first SaaS, then built both from scratch underneath: argon2 sessions, a double-entry ledger with a money race test, a transactional outbox with a dead-letter queue, self-hosted on a VPS. The last project is a streaming AI avatar under two seconds end-to-end. Everything has tests in CI and a written trade-offs section, and I document how I use AI tools and where they were wrong.

**8.3 Guaranteed questions — have crisp answers** - JS: event loop & microtasks · closures · `this` · prototypes vs classes · `==` vs `===` /coercion · promises vs `async/await` · `debounce/throttle` from scratch. - TS: generics, utility types, `unknown` vs `any`, discriminated unions, typing props/hooks/events. - React: reconciliation & keys · `useEffect` lifecycle & cleanup · lifting vs colocating state · `memo/useMemo/useCallback` (when *not* to) · Context vs store · Server Components vs client · data-fetching states. - Web: what happens when you type a URL · how the browser renders · HTTP/1.1 vs 2 vs 3 · caching headers · CORS · cookies vs tokens · CSRF/XSS. - Backend/DB: REST design & status codes · idempotency · rate limiting · indexes & when they hurt · transactions & isolation levels · N+1 · keyset pagination · webhook at-least-once. - Systems: outbox pattern · DLQ · retries/backoff · reconciliation · circuit breaking · eventual consistency · why 2PC died · saga at scale. - AI: how an LLM works (Karpathy depth) · RAG pipeline & failure modes · streaming/backpressure · evals · structured outputs · what MCP is.

**8.4 Portfolio walkthrough script (per project, 2 minutes each):** problem → architecture in one breath → the one hard decision and its rejected alternative → the never-cut test → the metric or artefact (latency bench, EXPLAIN table, CI badge) → what you'd do next. Rehearse aloud; the tracker's Deep Work labels are the outline.

**8.5 Take-home rules.** Read the rubric twice. Timebox to what they asked. Tests on the critical path, a README with run instructions, decisions and limitations. Ship on time over shipping perfect. Never over-engineer; do note what you *would* add.

**8.6 System design method (45 min):** clarify requirements & scale (5) → API + data model (10) → high-level design (10) → deep dive on the two hardest parts (15) → trade-offs & what you'd monitor (5). Anchor every answer in something you actually built.

**8.7 Questions to ask them.** How do you deploy and roll back? What broke last quarter? How is AI used on the team, and who reviews it? What does a mid-level engineer own here in six months?

**8.8 Cadence in the six weeks:** LeetCode 30 min/day (Tier 2) · one system-design mock/week (record yourself) · one live-component rep/week · re-read [react.dev Quick Start](#) and the TS Handbook generics chapter · certs sat in this window (§3.3) · profile README, pinned repos and blog groomed.

## 9. How to use the tracker (the short manual)

---

- **Sessions are the unit.** Every roadmap task = N sessions (~2 h each). Click to log one, `-` to undo, `+plan` if it's taking longer than estimated. Complete when logged  $\geq$  planned.
  - **Sunday:** declare DEFAULT / REDUCED / FALLBACK from your real calendar. The week generates the next sessions top-down from the roadmap (plus keep-warm and passive items in lower modes).
  - **During the week:** log as you go, here or in the Roadmap tab — both count. Finished the plan? *Pull next*  $\rightarrow$ . Passive items tick = one unit logged. Change mode mid-week if needed.
  - **It closes itself** on the next Sunday and records the sessions. Sessions logged without declaring still count. Weeks with nothing count as 0.
  - **Projection:** velocity = average of (sessions  $\div$  DEFAULT budget) over the last 8 weeks. Two levers pull dates in: log more sessions, or cut scope. After four weeks well over/under budget it suggests changing the DEFAULT budget (Data  $\rightarrow$  Budgets).
  - **Daily:** log the drill (pull  $\rightarrow$  sync  $\rightarrow$  quiz  $\rightarrow$  answer live  $\rightarrow$  commit) and bump commits in Threads.
  - **Data:** cloud sync via your private gist keeps devices aligned; export JSON monthly anyway. Full details in the tracker's README.
-

## 10. Provisional timeline — the target to try for

The tracker owns the real dates; this table is the plan you're trying to beat. It assumes DEFAULT (4 sessions/week) most weeks with event-season dips, i.e. an effective velocity around 0.75, and today's session estimates. Re-snapshot it [here](#) whenever the projection changes materially (suggested: 1 December 2026, and at every project ship).

| Phase                                                      | Sessions left | Target window                |
|------------------------------------------------------------|---------------|------------------------------|
| React + TypeScript (Jonas parts 9 → end, Next.js, TS, RTL) | ~34           | Aug → mid-Nov 2026           |
| Project 1 · Portfolio + first blog post                    | ~11           | mid-Nov → mid-Dec 2026       |
| Backend reading                                            | 3             | mid-Dec 2026                 |
| Node.js + Express + Testing (with Postgres/Prisma detour)  | ~18           | mid-Dec 2026 → end Jan 2027  |
| Databases (parallel,    0.7)                               | ~9            | Jan 2027                     |
| Project 2 · NFC Platform ★                                 | ~27           | Feb → early Apr 2027         |
| Soft applications open                                     |               | ≈ early April 2027           |
| Project 3 · StockSense Deep (   0.6, overlaps NFC tail)    | ~17           | Mar → Apr 2027               |
| Project 4 · Commerce Engine ★                              | ~22           | Apr → mid-Jun 2027           |
| Project 5 · AI Avatar (   0.6, overlaps Commerce tail)     | ~19           | May → mid-Jul 2027           |
| Certs (AWS CLF-C02, AZ-900) — 1 hr/day, passive            | pool          | in the interview-prep window |
| Full job-search push                                       |               | ≈ mid-July 2027              |

**Checkpoint — 1 December 2026:** compare this table with the tracker's projection. If React slipped, shift the table honestly rather than compressing projects. If compression is ever forced, cut in the order in \$0.

Every DEFAULT week you beat (5–6 sessions) pulls both dates in; every undeclared week pushes them out. The tracker shows the truth weekly.

## 11. Principles — read when tempted to deviate

- No skipping fundamentals. C# experience doesn't exempt you from JS coercion, hoisting, `this`, prototypes — you've been right every time you refused shortcuts.
- Projects are the plan, not extras. Skipping them to go faster weakens the foundation.
- Depth beats count. Three deep, tested, documented projects > five shallow ones. Cut order in \$0.
- AI fluency is a marketable skill, not cheating — but only when paired with validation. Use the tools, audit the output, publish the audit.
- Tests and validation are how a mid-level engineer looks mid-level. Nobody ever complained about a green CI badge.
- The public footprint compounds. A commit and a paragraph today beat a scramble next spring.
- A bad week happens. Don't let it become a bad month. Declare REDUCED, keep warm, recalibrate at check-ins instead of silently compressing.
- The plan is fluid; the standards are not.

## 12. Quick links index

---

**Courses (Udemy):** [Git](#) — [Colt Steele](#) · [JS](#) — [Jonas](#) · [React](#) — [Jonas](#) · [Node](#) — [Jonas](#) · [DB Engineering](#) — [Nasser](#) · [AWS CLF-C02](#) — [Maarek](#) · [AWS practice](#) — [Bonso](#) · [AZ-900](#) — [Duffy](#)

**References:** [javascript.info](#) · [react.dev](#) · [TS Handbook](#) · [Next.js](#) · [Tailwind](#) · [Node.js](#) · [Express](#) · [Prisma](#) · [PostgreSQL](#) · [PostgreSQL Tutorial](#) · [Use The Index, Luke](#) · [Supabase](#) · [Neon](#) · [Redis](#) · [MDN](#) · [roadmap.sh/full-stack](#) · [roadmap.sh/backend](#)

**Testing & automation:** [Jest](#) · [Vitest](#) · [React Testing Library](#) · [Supertest](#) · [Playwright](#) · [Zod](#) · [React Hook Form](#) · [GitHub Actions](#) · [Husky](#) · [lint-staged](#) · [Dependabot](#) · [Testing Trophy](#) · [Conventional Commits](#)

**Projects stack:** [better-auth](#) · [Stripe docs](#) · [Payment Intents](#) · [Stripe webhooks](#) · [BullMQ](#) · [Socket.io](#) · [pgvector](#) · [Modal](#) · [React Three Fiber](#) · [DigitalOcean docs](#) · [nginx](#) · [OWASP Cheat Sheets](#) · [OWASP Top 10](#) · [Docker](#)

**System design & AI:** [Hussein Nasser YT](#) · [systemdesignschool.io](#) · [ByteByteGo](#) / [Alex Xu](#) · [Karpathy](#) — [Intro to LLMs](#) · [Karpathy](#) — [Deep Dive](#) · [3Blue1Brown](#) · [Anthropic prompting](#) · [promptingguide.ai](#) · [MCP](#) · [Claude Code](#) · [Simon Willison](#) · [roadmap.sh/ai-engineer](#)

**Certs — free:** [AWS Skill Builder](#) · [Microsoft Learn AZ-900](#) · [MeasureUp](#) · [Tutorials Dojo](#)

**Writing & footprint:** [dev.to](#) · [Hashnode](#) · [DevJamaly/guides-and-cheatsheets](#) · [GitHub profile README guide](#)

**Practice & jobs:** [LeetCode](#) · [Arc.dev](#) · [Turing](#) · [Wellfound](#) · [We Work Remotely](#) · [Himalayas](#) · [HN Who's Hiring](#) · [Bayt](#) · [GulfTalent](#) · [LinkedIn Jobs](#) · [Meetup Dubai](#) · [GITEX](#)

**Your tools:** [journey-tracker](#) (your deployed URL) · [Cloud sync setup](#) — [tracker README](#)